Exercise 1: First Conversations

Session: 1 — First Steps & Core Concepts
Duration: 25 minutes
Project: "BookStore API" — a Go REST API using only `net/http`. Everyone clones the same repo.

Goal

You have your first conversations with Claude in your editor. In this exercise you:

1. Ask Claude to explain code, and see how many steps one answer takes.
2. Let Claude find a bug, add a check, and write a test.
3. Read the trace after each task, and write down what Claude did.

Start here

- Open the `bookstore` folder in your editor. Start `claude` in its terminal.
- Training mode is on. The file `CLAUDE.md` tells Claude to ask you a question before it helps. Answer it. Stuck? Type *just tell me*.
- Never edit `CLAUDE.md`.
- Write your notes on paper or in a text file outside the project.

Tasks

1. Explain, and count the work behind the answer

This task shows you how many steps one answer takes.

1. **Open the trace.** `/xtrace` is a command that came with this project. Run it:

```
/xtrace tools model pane
```

   A pane opens next to the chat. It adds one `tools` row for each tool call: a search, a read, an edit or a command. A `model` row is one cycle: one call to the model. A turn is your message and everything Claude does until it answers. No pane opens? Then run `/trace` after each answer. It lists the tool calls of the last turn.

2. **Predict.** Write two numbers: how many tool calls and how many cycles will Claude need to explain `Search()`?

3. **Ask.** Paste both lines. The first line is a tag. It switches training mode off, so Claude answers at once.

```
[Exercise 1 experiment — execute directly, no leading questions.]
Explain step by step what Search() in the book store does
```

4. **Count.** Read the answer. Count the `tools` rows and the `model` rows in the pane, and write both numbers down. Does the answer say the code runs one extra database query for each book? Write down yes or no.

5. **Look.** Which tool did Claude use first, and which file did it open?

Claude cannot see the code until a tool call opens it. Every round of tool calls is one more cycle.

Done when: you wrote your prediction and both counts.

2. Find a bug

This task shows you which files Claude opens to find a bug.

1. **Clear.** Run `/clear`.

2. **Predict.** Write one sentence: which other file will Claude open to answer?

3. **Ask.** Paste this prompt:

```
Is there a bug in CreateReview() in internal/handler/review.go?
```

4. **Answer.** Training mode makes Claude ask you a question first. Answer it. Then read Claude's answer.

5. **Step back.** The pane shows the last turn. Press ctrl+x, then Tab, so the pane reads your keys. Press `p` to see the turn before. Press Esc to type in the chat again. With `/trace`, run `/trace all` instead. Which file did Claude open, and in which turn? Was your prediction right?

Done when: Claude named a bug, and you can say in one sentence what `CreateReview()` fails to check.

3. Add validation, say no, add a test

This task changes `internal/handler/review.go` and creates `internal/handler/validate_test.go`.

1. **Clear.** Run `/clear`. Then paste this prompt:

```
Add input validation to CreateReview: rating must be 1-5, review_text must be between 10 and 500 characters
```

2. **Look.** Answer Claude's question, if it asks one. Then run `git diff`. Where did Claude put the checks? In the handler, in a separate function, or in struct tags?

3. **Say no.** Paste this prompt:

```
No, extract the validation into a separate validateReview() function instead
```

4. **Add a test.** Paste this prompt:

```
Write a table-driven test for validateReview() covering: rating 0, rating 6, text of 9 characters, text of 501 characters, and one valid review. Put it in internal/handler/validate_test.go, in package handler, and name it TestValidateReview
```

5. **Run the test.** Run this command yourself:

```
go test ./internal/handler/ -run TestValidateReview -v
```

   Other tests in this package fail on purpose. Session 2 deals with them. Look only at the five subtests.

6. **Read the pane.** Step through the turns: ctrl+x, Tab, then `p` (back) and `n` (forward). Which tool changed the file? Did Claude run anything to check its change? Did Claude run the test, or only you? If Claude ran nothing, write that down.

Saying no is normal. You decide what goes into your code.

Done when: `validateReview()` exists in `internal/handler/review.go`, `CreateReview()` calls it, and the test command shows five passing subtests.

4. Closing: reset, then ask a big question

This task gives you two numbers about the whole API.

1. **Clear.** Run `/clear`. Claude now has no memory of tasks 1 to 3. You still have your notes.
2. **Ask.** Paste this prompt:

```
What HTTP status codes does this API return, and which ones violate REST conventions?
```

3. **Count.** In the pane, count the files Claude opened. Count each file once. In its answer, count the status-code problems. Count each problem once.

Done when: you wrote two numbers: files opened, and status-code problems found.

Bonus (only if time remains)

Ask the question from task 4 again, in the same conversation. Press ctrl+x, Tab, then `p` to compare with the first turn. Did Claude need fewer tool calls? Say why in one sentence.

Closing round

The trainer calls on people at random. Have your numbers ready:

- How many tool calls and cycles for the `Search()` question?
- Which file did Claude open first for the bug? Was your prediction right?
- Did Claude run the test, or only you?
- How many files for the status-code question, and how many problems?